environment and the command line environment. Here is a list of a few:

1. Vi or Vim
2. Emacs
3. Nano
4. Gedit
5. kwrite

Now, we can type in the first script using any of the above text editors, as follows:

```
#!/bin/bash
# My first script
echo "Hello World!"
```

The above script has three lines.

- The first line is a special clue given to the shell in order to indicate what program is used to interpret this script. In this case, its */bin/bash*. Other scripting languages such as Awk, Perl, Tcl, Tk, and Python can also use this type of mechanism.
- The second line is just a comment. Everything that comes after a '#' symbol is all ignored by bash. As our scripts become larger and more complicated, comments become important. They are basically used by programmers to explain what's going on, so that others can understand.
- The last line is the *echo* command, which just prints whatever is given on the display. Once we have written the above script, we can save this file using any name. Here, we save it as *First_Shell_Script*.

2. *Give the shell permission to execute it*

The next thing we need to do is to give the shell permission to execute our script. This is done using the *chmod* command, as follows:

```
[me@linuxbox me]$ chmod 755 my_script
```

The '755' will give us permission to read, write and to execute. Everybody else will just have read and execute permission. If we want our script to be private (so that only we can read and execute), we use '700' instead.

3. *Put it at the location where the shell can find it*

We now need to run our script by typing the following command:

```
[me@linuxbox me]$ ./First_Shell_Script
```

Now, you should see 'Hello World!' displayed. If we don't, then we need to check what directory we saved our script in, so that we can go there and try again.

Before we go further, let's discuss paths a little bit. When we type in the name of a command, our system does not search the entire computer in order to find where the program is located. That would really take a long time. You will have noticed that we don't usually have to specify the complete path name to the program we want to run; the shell just seems to know that. The shell maintains a list of different directories where executable files are kept, and it just searches for the directories in that list. If it's not able to find the program after searching each of the directories in the list, it will throw up the error: *Command not found*.

This list of directories is called the *path*. Let us look at some of the commands related to this along with their possible output.

- The command for viewing the list of directories is as follows:

```
[me@linuxbox me]$ echo $PATH
```

The output: A colon-separated list of directories which will be searched if a specific path is not given when a command is attempted.

- The command for adding directories to the path, where *directory_name* is the name of directory we want to add, is:

```
[me@linuxbox me]$ export PATH=$PATH:directory_name
```

The output: *directory_name* will be added to the path.

- The command for creating a *bin* directory, which is a sub-directory of our home directory, is as follows:

```
[me@linuxbox me]$ mkdir bin
```

The output: The bin directory will be created.

Shell scripting can be used in the following cases:

1. To create our own command or tool.
2. To monitor several tasks like disk usage, etc.
3. To take data backup and snapshots.
4. For automatic system booting.
5. For automatic installation of different application packages.
6. To automate different aspects of computer maintenance and user account creation.
7. To create startup scripts for different unattended applications.
8. To kill or start multiple applications together. END

References

[1] *http://www.wikipedia.org/*
[2] *http://www.guru99.com/*
[3] *http://linuxcommand.org*

By: Vivek Ratan

The author currently works as an automation test engineer at Infosys, Pune. He can be reached at *ratanvivek14@gmail.com* for any suggestions or queries.